

# **Laboratory Activity No. 1**

## **Real-Time Multisensor Room Monitoring System**

**Course:** BCA152 Microcontrollers

**Institution:** Mindanao State University - Iligan Institute of Technology (MSU-IIT)

**College / Department:** College of Computer Studies / Department of Computer Applications

**Instructor:** Asst. Prof. Paul Rodolf P. Castor, M.Sc.

**Author / Student:** Yestin Paraguya

**Date:** September 20, 2026

**Repository:** [xavieryyyyyy/esp32-freertos-room-monitoring-system](https://github.com/xavieryyyyyy/esp32-freertos-room-monitoring-system)

# 1. Problem and Requirements

## 1.1 Project Overview and Context

In everyday facilities such as server rooms, hospital medicine refrigerators, and agricultural greenhouses, environmental conditions must stay within safe limits. A server room that overheats can shut down abruptly and ruin expensive computers. A medicine cabinet that drops below freezing can ruin insulin and vaccines. To prevent these problems, facilities rely on continuous room monitoring devices that read temperature, humidity, light, and room motion, sounding an alarm the moment conditions become unsafe.

### The Technical Problem:

Most beginner embedded projects run inside a single loop, often called a super-loop or while(1) structure. In that model, the processor executes one instruction after another in sequence. It reads sensor data, waits, updates a screen, waits, and checks an alarm threshold.

This structure fails when peripherals require long communication delays. For example, reading a DHT22 digital sensor requires the microcontroller to time electrical pulses on a single wire for up to 25 milliseconds. Refreshing a graphics screen like an SSD1306 OLED over an I2C bus takes another 10 to 15 milliseconds. If a person turns a control knob or walks in front of a motion sensor while the processor is busy waiting inside one of these routines, the processor misses the electrical signal completely. The device feels unresponsive and can fail to record important events.

### The Solution:

This project solves that limitation by building a concurrent embedded system on an ESP32 microcontroller using FreeRTOS under the native ESP-IDF framework. Instead of forcing one long loop to handle every chore, the software is divided into separate, self-contained tasks. A real-time scheduler gives CPU time to whichever task has the most urgent work, pausing slower background routines whenever immediate user input or motion occurs.

## 1.2 Required System Behavior

The system fulfills ten functional requirements specified in the laboratory guidelines:

- **FR-01 (Temperature measurement):** The system reads room temperature from a digital sensor every 2.0 seconds.
- **FR-02 (Humidity measurement):** The system reads relative humidity from the same sensor every 2.0 seconds.
- **FR-03 (Ambient light measurement):** The system reads a photoresistor through an analog pin and converts raw voltage readings into a simple 0% to 100% relative scale.
- **FR-04 (Motion detection):** The system monitors a passive infrared (PIR) sensor to detect human presence.
- **FR-05 (OLED display output):** The system displays one telemetry item at a time on an OLED screen with clear labels and units.

- **FR-06 (Rotary encoder navigation):** The user can rotate a physical knob clockwise or counterclockwise to cycle through four display screens (Temperature, Humidity, Light, and Motion) with continuous wraparound in both directions.
- **FR-07 (Temperature alarm):** A piezo buzzer sounds a 2000 Hz tone whenever the temperature drops below 18.0 °C or rises above 30.0 °C. The buzzer silences when the temperature returns between 18.0 °C and 30.0 °C.
- **FR-08 (Activity states):** The system operates in an ACTIVE state during normal room activity and switches to an INACTIVE state when no motion is observed.
- **FR-09 (Automatic inactivity timeout):** If the PIR sensor detects no motion for 15.0 continuous seconds, the system enters the INACTIVE state and clears the OLED screen to save energy and protect display hardware.
- **FR-10 (Automatic motion wake-up):** As soon as the PIR sensor detects motion, the system switches back to ACTIVE and restores the telemetry display without delay.

### 1.3 Simulation Environment Setup

Hardware components can be simulated inside software tools before assembling physical circuit boards. For this project, the Wokwi embedded simulator is used, which accurately models the timing, electrical signals, and processor instructions of real components.

The simulated circuit uses these components:

- ESP32 DevKit-C v4 development board with a dual-core 240 MHz processor and 4MB of flash storage.
- DHT22 digital temperature and humidity sensor attached to GPIO 4.
- LDR photoresistor module attached to GPIO 34 (ADC1 Channel 6) in a voltage divider circuit.
- SSD1306 128×64 pixel OLED display connected over an I2C serial bus on GPIO 21 (SDA) and GPIO 22 (SCL).
- KY-040 rotary encoder connected to GPIO 18 (clock pin) and GPIO 19 (data pin).
- Piezoelectric buzzer attached to GPIO 25 driven by hardware pulse width modulation (PWM).
- PIR motion sensor attached to GPIO 27.

## 2. System Architecture and Design

### 2.1 Hardware Architecture and Connections

The table below lists every peripheral, its wiring to the ESP32, the communication protocol used, and its specific purpose in the system:

| Peripheral            | ESP32 Pin                  | Communication Type          | Role in the System                                                                         |
|-----------------------|----------------------------|-----------------------------|--------------------------------------------------------------------------------------------|
| DHT22 Sensor          | GPIO 4                     | 1-Wire Digital Protocol     | Transmits digital temperature and humidity data using microsecond-timed electrical pulses. |
| Photoresistor (LDR)   | GPIO 34                    | Analog (ADC1 Channel 6)     | Produces an analog voltage proportional to room light level.                               |
| SSD1306 OLED Display  | SDA: GPIO 21, SCL: GPIO 22 | I2C Bus (0x3C, 400 kHz)     | Displays telemetry values and alarm status on a 128×64 dot-matrix screen.                  |
| KY-040 Rotary Encoder | CLK: GPIO 18, DT: GPIO 19  | Hardware Pin Interrupts     | Generates quadrature square-wave pulses when the user turns the knob.                      |
| Piezo Buzzer          | GPIO 25                    | Hardware PWM (LEDC Timer)   | Generates a 2000 Hz square wave to sound the audible temperature alarm.                    |
| PIR Motion Sensor     | GPIO 27                    | Digital Input (Active High) | Outputs 3.3V when human movement is sensed and 0V when quiet.                              |

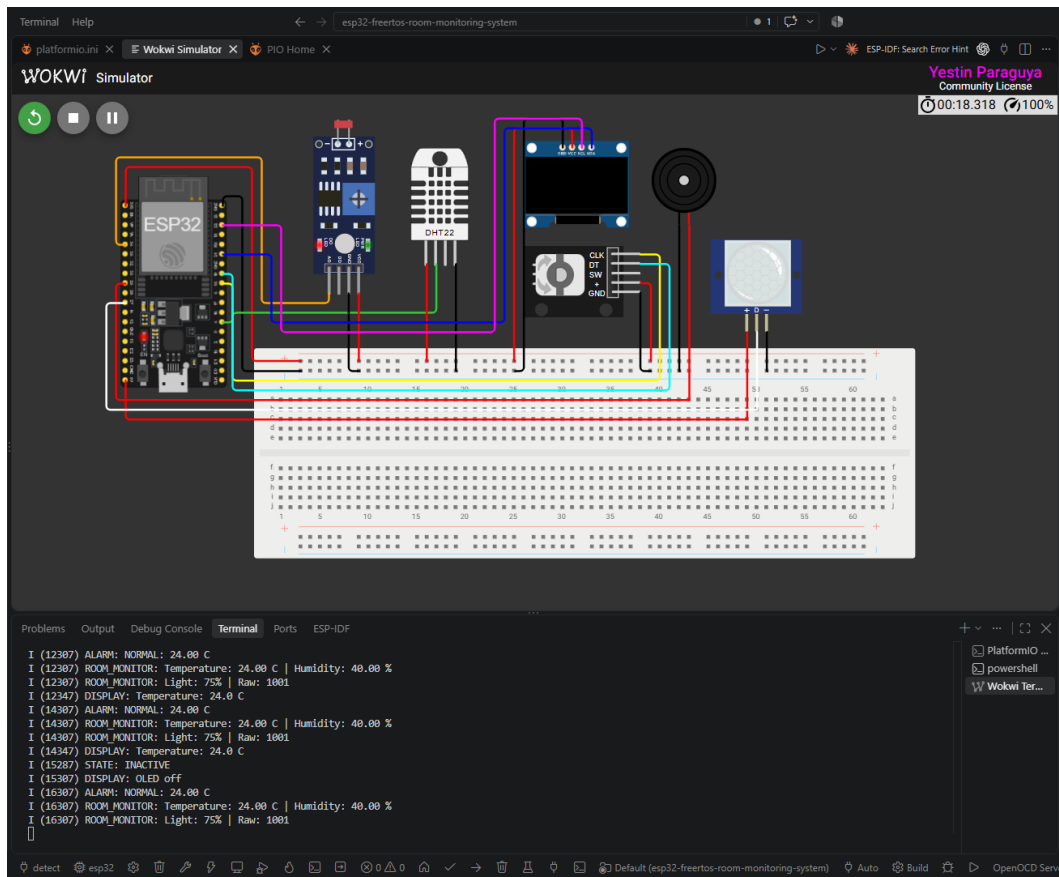


Figure 1: Wokwi circuit schematic showing ESP32 DevKit-C pin connections to all six peripheral modules.

## Explanation of communication types:

1. **1-Wire Digital:** The ESP32 pulls the single data wire low to start a reading, and the DHT22 responds with 40 bit pulses. Each pulse width indicates whether a bit is a zero or a one.
2. **Analog to Digital Converter (ADC):** The photoresistor acts as a variable resistor. As room light changes, the voltage on GPIO 34 shifts between 0V and 3.3V. The internal 12-bit ADC converts this continuous voltage into an integer between 0 and 4095.
3. **I2C Bus:** A two-wire serial interface sharing a clock line (SCL) and a data line (SDA). It allows the ESP32 to send bitmap graphics to the display controller without taking up many pins.
4. **Hardware Interrupts:** Instead of having software constantly check the encoder pins, the ESP32 hardware triggers an interrupt service routine the exact microsecond a knob pin changes voltage.
5. **Hardware PWM (LEDC):** The ESP32 LED control peripheral uses an internal hardware counter to toggle the buzzer pin at 2000 cycles per second. This produces sound without using CPU software cycles.

## 2.2 Software Structure and Organization

In well-built software, code that talks directly to physical wires should not be mixed with calculation logic or decision rules. If you mix them together, you cannot test your business logic on a regular computer without attaching physical hardware.

To keep the system organized and maintainable, the source code is divided into three distinct layers:

6. **Hardware Driver Layer (sensors.cpp, display.cpp, input.cpp, motion.cpp, alarm.cpp):** These files contain the direct calls to ESP-IDF hardware functions. They initialize GPIO pins, configure the ADC, set up I2C transactions, manage hardware interrupt handlers, and start PWM timers.
7. **FreeRTOS Synchronization Layer (rtos\_objects.cpp, main.cpp):** These files create and connect the operating system primitives. They initialize memory queues, event flags, and mutex locks, launching the independent tasks with assigned stack sizes and priority levels.
8. **Pure Decision Logic Layer (alarm\_logic.cpp, display\_mode.cpp, system\_state.cpp):** These files contain pure computational functions. They do not include any ESP-IDF headers, hardware pin definitions, or FreeRTOS libraries. For example, evaluateTemperature() accepts a raw floating-point number and returns an enumerated alarm condition (ALARM\_OK, ALARM\_LOW, or ALARM\_HIGH). Because this layer has zero hardware dependencies, it is compiled and tested on a standard developer computer using automated unit test runners.

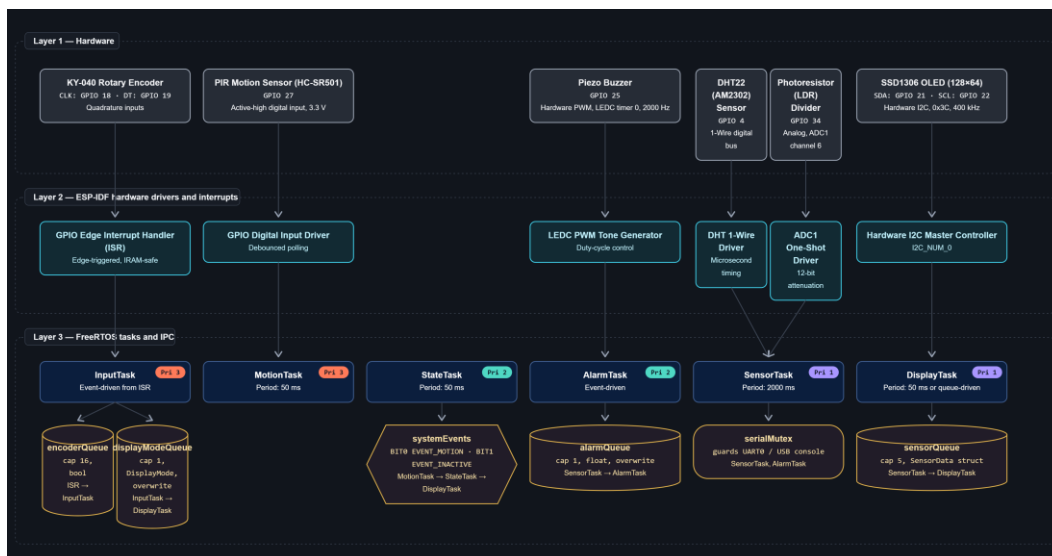


Figure 2: Three-layer system architecture showing physical sensors, ESP-IDF drivers, and FreeRTOS tasks with IPC queues.

## 2.3 System State Machine

A state machine is a design pattern where an application lives in one of several clearly defined modes, called states. The system responds to incoming events differently depending on which state is currently active, transitioning between states when specific conditions are met.

This system implements a two-state machine to balance user visibility with power conservation:

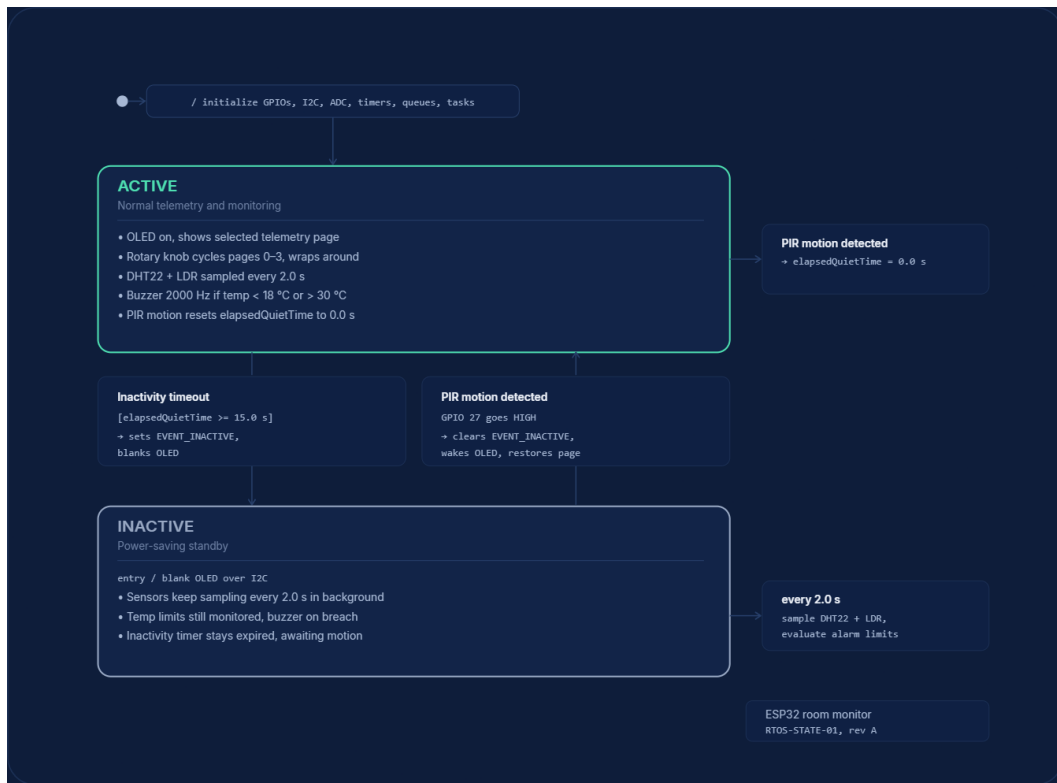


Figure 4: Energy management state machine transition diagram showing *ACTIVE* and *INACTIVE* states.

### State Details and Transition Behavior:

- **ACTIVE state:** The OLED display is powered on. Telemetry data updates on the screen every 50 milliseconds or whenever new sensor data arrives. The rotary encoder actively rotates between screens, and the temperature alarm sounds if limits are breached.
- **INACTIVE state:** When 15.0 seconds elapse without the PIR motion sensor detecting activity, the system transitions to INACTIVE. The display task clears the OLED screen buffer and sends a blank image to the screen. The background sensors continue collecting data and the alarm remains ready to sound if temperature limits are crossed.
- **Transition from INACTIVE back to ACTIVE:** As soon as the PIR sensor detects motion, an event flag is raised. The state task switches the operating mode back to ACTIVE, the display task repaints the active telemetry page, and the 15-second inactivity countdown resets to zero.

## 3. FreeRTOS Architecture

### 3.1 Task Breakdown

Instead of executing sequentially in a super-loop, system operations are split across six independent FreeRTOS tasks. The table below outlines their individual responsibilities, execution schedules, priority settings, and synchronization objects:

| Task Name   | Primary Responsibility                | Trigger / Period        | Priority | IPC Mechanisms                              | Typical Blocked Condition                                          |
|-------------|---------------------------------------|-------------------------|----------|---------------------------------------------|--------------------------------------------------------------------|
| SensorTask  | Reads DHT22 and LDR sensors           | Periodic (2000 ms)      | 1        | sensorQueue, alarmQueue, serialMutex        | Blocked on vTaskDelayUntil() waiting for next 2-second timestamp.  |
| DisplayTask | Manages and draws the OLED screen     | Event or 50 ms periodic | 1        | sensorQueue, displayModeQueue, systemEvents | Blocked on sensorQueue waiting for incoming sensor telemetry.      |
| AlarmTask   | Evaluates temperature and runs buzzer | Event-driven            | 2        | alarmQueue, serialMutex                     | Blocked on alarmQueue waiting for new temperature packet.          |
| StateTask   | Evaluates 15s inactivity timeout      | Periodic (50 ms)        | 2        | systemEvents, serialMutex                   | Blocked on vTaskDelayUntil() waiting for next 50 ms tick interval. |
| MotionTask  | Samples PIR pin and posts event bits  | Periodic (50 ms)        | 3        | systemEvents                                | Blocked on vTaskDelay() waiting for next 50 ms sample interval.    |
| InputTask   | Processes encoder pulses from ISR     | Event-driven            | 3        | encoderQueue, displayModeQueue              | Blocked on encoderQueue waiting for ISR to push rotation step.     |

### 3.2 Scheduling and Priority Justification

In FreeRTOS, each task is assigned an integer priority number. The FreeRTOS scheduler uses a priority-based preemptive scheduling algorithm. This means the CPU will always run the highest-priority task that is ready to work. If a higher-priority task wakes up while a lower-priority task is running, the scheduler immediately pauses the lower-priority task and hands CPU control to the higher-priority one.

Priorities are assigned based on task deadlines, execution durations, and user responsiveness:

- **Priority 3 (Highest: InputTask and MotionTask):** These tasks handle brief physical events. When a user turns the rotary knob, the metal contacts produce a voltage pulse that lasts only a

few milliseconds. If the processor does not process that pulse immediately, the pulse passes and the user experiences an unresponsive knob. Similarly, motion detection pulses must be recorded without lag. Because both tasks execute in less than 0.1 milliseconds before going back to sleep, giving them Priority 3 guarantees an instant response without hogging CPU time.

- **Priority 2 (Medium: AlarmTask and StateTask):** These tasks manage system safety and state transitions. When temperature data arrives, AlarmTask must decide within a reasonable window whether to sound the buzzer. StateTask tracks the 15-second inactivity timeout. Both tasks perform simple numerical checks that take less than 0.5 milliseconds, so they wake up, execute, and block without delaying sensor reads.
- **Priority 1 (Lowest: SensorTask and DisplayTask):** These tasks perform long, slow operations. Updating the OLED display requires transmitting over 1000 bytes across the I2C bus, taking 10 to 15 milliseconds. Sampling the DHT22 requires timing pulses for up to 25 milliseconds. If DisplayTask were given a high priority, that 15-millisecond screen update would lock out the rotary encoder, causing the system to miss knob clicks. Keeping DisplayTask and SensorTask at Priority 1 ensures that high-priority user actions can preempt screen rendering at any moment.

### 3.3 FreeRTOS Task States

At any point in time, a FreeRTOS task exists in one of four operating states:

9. **Running:** The task is currently occupying the CPU core and executing instructions. On a single core, only one task can be Running at any given instant.
10. **Ready:** The task is prepared to execute code, but it cannot run yet because another task of equal or higher priority is currently occupying the CPU.
11. **Blocked:** The task is waiting for a specific event or time interval to occur. While a task is Blocked, it consumes zero percent of CPU processing time. It wakes up only when its timer expires or data arrives in its queue.
12. **Suspended:** The task is explicitly put on hold using `vTaskSuspend()` and will not run until another task calls `vTaskResume()`. The design does not require the Suspended state.

**How system tasks transition through these states:**

- **SensorTask:** Executes its reading loop (Running), pushes readings into queues, and calls `vTaskDelayUntil()`, which places it into the Blocked state for roughly 1970 milliseconds. When the timer tick reaches 2000 milliseconds, the scheduler moves it to Ready. Because its priority is 1, it enters Running once higher-priority tasks complete.
- **InputTask:** Spends almost all its time Blocked on `xQueueReceive(encoderQueue, ...)`. When the user turns the knob, the hardware interrupt service routine posts a direction code into the queue, shifting InputTask to Ready. Because InputTask is Priority 3, the scheduler immediately preempts any Priority 1 or 2 task and moves InputTask to Running.
- **AlarmTask:** Stays Blocked waiting for an item in `alarmQueue`. When SensorTask writes a temperature reading, AlarmTask unblocks, evaluates the limits, controls the buzzer pin, and blocks again.

### 3.4 Periodic Execution: vTaskDelayUntil() vs vTaskDelay()

When writing periodic code, developers often use simple delay commands. However, simple delays cause timing errors that accumulate over time.

#### Comparison of delay behaviors:

- **vTaskDelay(2000):** This function pauses execution for 2000 milliseconds starting from the moment the function is reached. If the sensor reading code before the delay takes 30 milliseconds to complete, the total cycle length becomes  $30\text{ ms} + 2000\text{ ms} = 2030\text{ ms}$ . In each cycle, the execution slips by 30 milliseconds. Over an hour, this accumulative drift shifts the sampling schedule by nearly a full minute.
- **vTaskDelayUntil(&lastWakeTime, pdMS\_TO\_TICKS(2000)):** This function calculates the exact future tick count when the task should wake up next based on its previous wake-up timestamp. If a reading takes 30 milliseconds on one cycle and 15 milliseconds on the next, vTaskDelayUntil() adjusts its sleep duration to 1970 ms and 1985 ms respectively. The task wakes up at exact 2000-millisecond intervals (2.0s, 4.0s, 6.0s, 8.0s), eliminating accumulative drift.

#### Code evidence in src/sensor\_task.cpp:

```
TickType_t lastWakeTime = xTaskGetTickCount();
const TickType_t periodTicks = pdMS_TO_TICKS(2000);

while (true) {
    // Perform sensor readings
    readSensors(&data);

    // Send to queues
    xQueueSend(sensorQueue, &data, 0);
    xQueueOverwrite(alarmQueue, &data.temperature);

    // Absolute periodic delay: prevents cumulative drift
    vTaskDelayUntil(&lastWakeTime, periodTicks);
}
```

### 3.5 Inter-Task Communication and Synchronization

When multiple tasks run independently, they need safe mechanisms to pass messages and share hardware peripherals without corrupting memory or printing broken text.

#### 1. FreeRTOS Queues (sensorQueue, alarmQueue, encoderQueue, displayModeQueue):

- A queue is a thread-safe FIFO (first-in, first-out) memory buffer.
- Global variables are avoided because if one task writes to a multi-byte struct while another task reads it, the reader can get partially updated, corrupted numbers. This defect is called a data race or torn read.
- In FreeRTOS, reading from a queue removes the item (destructive read). If DisplayTask and AlarmTask shared a single queue, whichever task woke up first would take the temperature

reading, leaving the second task starved. Using separate queues guarantees both tasks receive the telemetry.

- alarmQueue holds a single item and uses xQueueOverwrite(). If the alarm task is momentarily busy, it does not need a backlog of old temperatures. It only needs the freshest value to decide whether to sound the alarm right now.

## **2. FreeRTOS Event Group (systemEvents):**

- An event group is a collection of individual binary flags (bits) stored in a single variable.
- BIT0 (EVENT\_MOTION): Set to 1 when the PIR sensor detects motion; cleared to 0 when motion stops.
- BIT1 (EVENT\_INACTIVE): Set to 1 by StateTask when 15 seconds pass without motion, signaling the display to go blank.
- Multiple tasks can read these bits concurrently without blocking one another or allocating extra message memory.

## **3. FreeRTOS Mutex (serialMutex):**

- A mutex (mutual exclusion lock) acts like a single physical key to a private room. Only the task holding the key can enter and use the shared resource.
- All tasks share a single physical UART serial port to print debug and status logs.
- If SensorTask is halfway through printing sensor values and AlarmTask interrupts to print an alarm message, the characters from both tasks interleave on the terminal screen, turning into unreadable garbage text.
- Surrounding all multi-line print statements with xSemaphoreTake(serialMutex, portMAX\_DELAY) and xSemaphoreGive(serialMutex) ensures that each task finishes its entire message before another task can print.

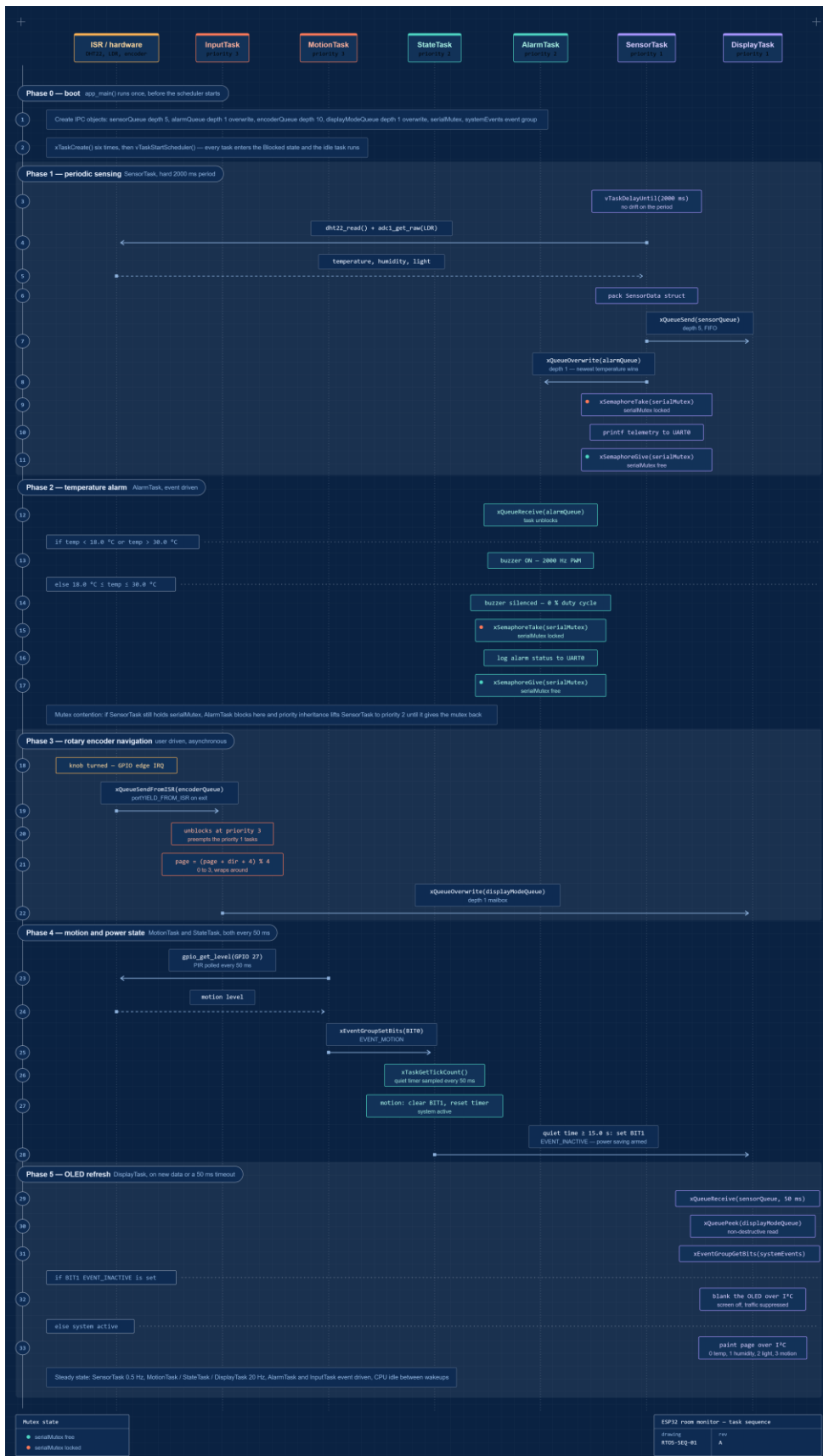


Figure 3: Chronological FreeRTOS task execution and communication sequence from boot, sensing, alarm evaluation, and screen refresh.



## 4. Implementation Details

### 4.1 System Entry Point (main.cpp)

Following embedded software engineering practices, `app_main()` contains no application loops, polling checks, or sensor calls. It functions strictly as an initialization orchestrator:

13. Prints the system banner to the serial console.
14. Initializes all FreeRTOS synchronization primitives (queues, mutexes, event groups) and verifies that memory allocation succeeded.
15. Initializes hardware drivers (GPIO pins, ADC, I2C, and PWM).
16. Creates all six FreeRTOS tasks with explicit stack depths and priority levels.
17. Completes its execution and returns. The underlying ESP-IDF runtime deletes the temporary startup task, leaving FreeRTOS in full control of all scheduling.

### 4.2 Module Responsibilities and Engineering Decisions

- **src/sensors.cpp:** Configures the DHT22 data pin and reads the 12-bit ADC on GPIO 34. The raw ADC reading (range 0 to 4095) is converted into a user-friendly light percentage:  $\text{Light Percentage} = (4095 - \text{raw\_adc}) \times 100 / 4095$ . This maps complete darkness to 0% and bright light to 100%.
- **src/display.cpp:** Initializes the SSD1306 display controller over the I2C bus at 400 kHz. It maintains an internal frame buffer and renders text based on the active page selected by the user.
- **src/input.cpp:** Attaches hardware edge interrupts to the rotary encoder pins. The interrupt service routine detects knob rotation direction and pushes the result into `encoderQueue` using `xQueueSendFromISR()`, keeping interrupt processing under 5 microseconds.
- **src/alarm.cpp:** Configures the ESP32 LEDC timer module to drive GPIO 25. When `AlarmTask` detects a temperature outside the 18.0 °C to 30.0 °C range, it sets the PWM frequency to 2000 Hz at a 50% duty cycle. When temperature normalizes, it sets the duty cycle to 0%, silencing the buzzer.
- **src/state\_task.cpp:** Samples tick time on every 50 ms cycle. If the elapsed time since the last recorded motion exceeds 15.0 seconds (15000 milliseconds), it raises `EVENT_INACTIVE` in `systemEvents`, triggering display shutdown.

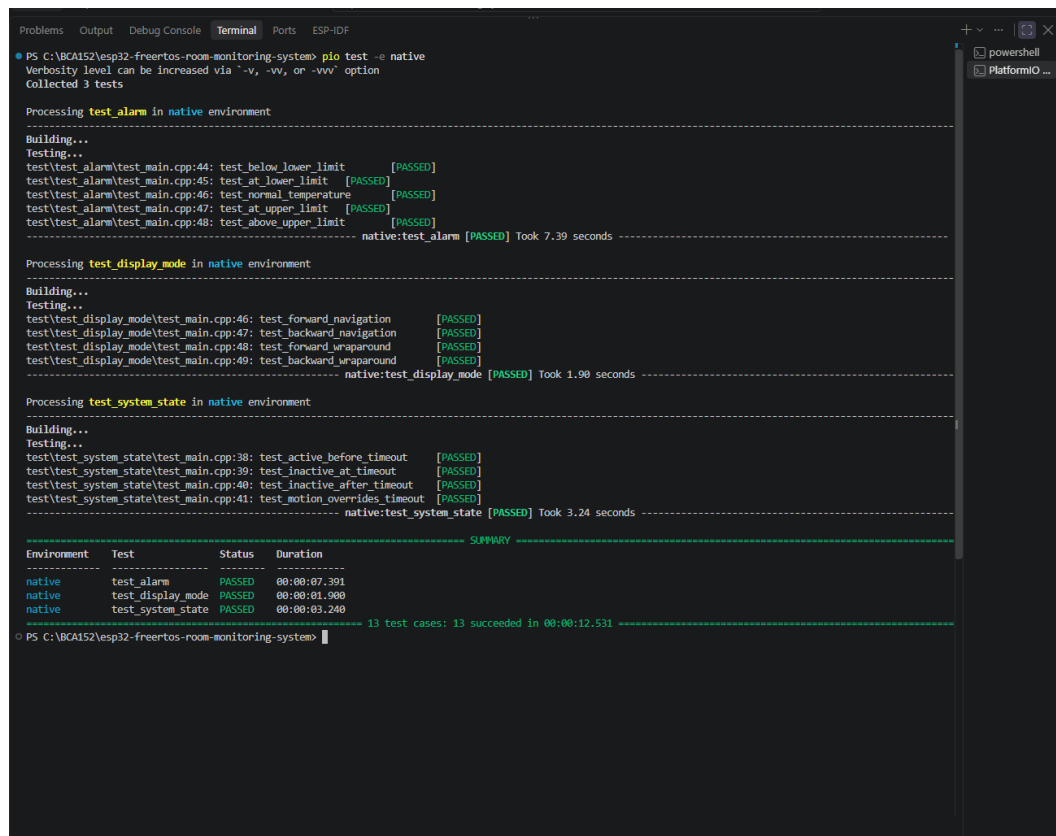
## 5. Verification and Testing

### 5.1 Automated Host Unit Tests (Unity Framework)

Before flashing firmware onto physical chips or simulators, automated unit testing runs isolated pieces of decision logic on the development computer. This confirms that all boundary conditions and math calculations function as expected.

Thirteen automated unit tests were implemented using the Unity testing framework in PlatformIO:

```
pio test -e native
```



```
PS C:\BCA152\esp32-freertos-room-monitoring-system> pio test -e native
Verbosity level can be increased via '-v', '-vv', or '-vvv' option
Collected 3 tests

Processing test_alarm in native environment
Building...
Testing...
test\test_alarm\test_main.cpp:44: test_below_lower_limit [PASSED]
test\test_alarm\test_main.cpp:45: test_at_lower_limit [PASSED]
test\test_alarm\test_main.cpp:46: test_normal_temperature [PASSED]
test\test_alarm\test_main.cpp:47: test_at_upper_limit [PASSED]
test\test_alarm\test_main.cpp:48: test_above_upper_limit [PASSED]
----- native:test_alarm [PASSED] Took 7.39 seconds -----

Processing test_display_mode in native environment
Building...
Testing...
test\test_display_mode\test_main.cpp:46: test_forward_navigation [PASSED]
test\test_display_mode\test_main.cpp:47: test_backward_navigation [PASSED]
test\test_display_mode\test_main.cpp:48: test_forward_wraparound [PASSED]
test\test_display_mode\test_main.cpp:49: test_backward_wraparound [PASSED]
----- native:test_display_mode [PASSED] Took 1.90 seconds -----

Processing test_system_state in native environment
Building...
Testing...
test\test_system_state\test_main.cpp:38: test_active_before_timeout [PASSED]
test\test_system_state\test_main.cpp:39: test_inactive_at_timeout [PASSED]
test\test_system_state\test_main.cpp:40: test_inactive_after_timeout [PASSED]
test\test_system_state\test_main.cpp:41: test_motion_overrides_timeout [PASSED]
----- native:test_system_state [PASSED] Took 3.24 seconds -----

----- SUMMARY -----
Environment  Test          Status  Duration
-----
native      test_alarm    PASSED  00:00:07.391
native      test_display_mode PASSED  00:00:01.900
native      test_system_state PASSED  00:00:03.240

----- 13 test cases: 13 succeeded in 00:00:12.531 -----
PS C:\BCA152\esp32-freertos-room-monitoring-system>
```

Figure 6: Automated Unity host unit test suite passing all 13 test cases in the native environment.

All 13 unit tests passed with zero failures:

| Test Suite | Function Tested         | Specific Scenario Tested                                                 | Result |
|------------|-------------------------|--------------------------------------------------------------------------|--------|
| test_alarm | test_below_lower_limit  | Temperature = 17.9 °C triggers low temperature alarm (ALARM_LOW).        | PASSED |
| test_alarm | test_at_lower_limit     | Temperature = 18.0 °C is classified as normal safe condition (ALARM_OK). | PASSED |
| test_alarm | test_normal_temperature | Temperature = 24.0 °C is classified as normal safe condition (ALARM_OK). | PASSED |

|                   |                               |                                                                          |        |
|-------------------|-------------------------------|--------------------------------------------------------------------------|--------|
| test_alarm        | test_at_upper_limit           | Temperature = 30.0 °C is classified as normal safe condition (ALARM_OK). | PASSED |
| test_alarm        | test_above_upper_limit        | Temperature = 30.1 °C triggers high temperature alarm (ALARM_HIGH).      | PASSED |
| test_display_mode | test_forward_navigation       | Clockwise turn steps through Temp, Humidity, Light, and Motion.          | PASSED |
| test_display_mode | test_backward_navigation      | Counterclockwise turn steps in reverse: Motion, Light, Humidity, Temp.   | PASSED |
| test_display_mode | test_forward_wraparound       | Advancing past Motion wraps smoothly back to Temperature.                | PASSED |
| test_display_mode | test_backward_wraparound      | Reversing past Temperature wraps smoothly back to Motion.                | PASSED |
| test_system_state | test_active_before_timeout    | At 14.99 seconds without motion, system remains in ACTIVE state.         | PASSED |
| test_system_state | test_inactive_at_timeout      | At exactly 15.00 seconds without motion, system transitions to INACTIVE. | PASSED |
| test_system_state | test_inactive_after_timeout   | Beyond 15.00 seconds without motion, system stays in INACTIVE.           | PASSED |
| test_system_state | test_motion_overrides_timeout | Presence of motion keeps the system ACTIVE even past 20 seconds.         | PASSED |

## 5.2 Wokwi Functional Verification (FT-01 through FT-10)

The complete application was deployed to the Wokwi simulator to verify that all physical components, buses, and FreeRTOS tasks interact correctly under user operation:

| Test ID | User Action or Input Stimulus               | Expected System Behavior                | Observed System Behavior                                                                                   | Status |
|---------|---------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------|--------|
| FT-01   | Adjusted DHT22 temperature slider in Wokwi. | OLED updates displayed temperature.     | Moving temperature to -5.2 °C and 40.5 °C updated the OLED numbers immediately on the next 2-second cycle. | PASS   |
| FT-02   | Adjusted DHT22 humidity slider in Wokwi.    | OLED updates displayed humidity.        | Humidity values updated smoothly across 40.0%, 42.5%, 77.0%, and 99.0% on screen.                          | PASS   |
| FT-03   | Adjusted photoresistor slider in Wokwi.     | OLED updates relative light percentage. | Raw ADC readings (1001, 3797, 226) mapped cleanly to 75%, 7%, and 94% on the display screen.               | PASS   |

|       |                                                          |                                          |                                                                                  |      |
|-------|----------------------------------------------------------|------------------------------------------|----------------------------------------------------------------------------------|------|
| FT-04 | Rotated rotary encoder clockwise.                        | Display switches forward to next page.   | Pages rotated cleanly: Temperature → Humidity → Light → Motion → Temperature.    | PASS |
| FT-05 | Rotated rotary encoder counterclockwise.                 | Display switches backward to prior page. | Pages rotated in reverse: Temperature → Motion → Light → Humidity → Temperature. | PASS |
| FT-06 | Configured temperature below 18 °C or above 30 °C.       | Buzzer sounds an audible alarm tone.     | At -5.2 °C and 40.5 °C, the buzzer produced an active 2000 Hz tone on GPIO 25.   | PASS |
| FT-07 | Returned temperature to safe range (21.4 °C).            | Buzzer immediately turns off.            | Buzzer silenced instantly; serial log output confirmed ALARM: NORMAL.            | PASS |
| FT-08 | Triggered PIR sensor motion in simulation.               | System stays in ACTIVE state.            | Serial log output confirmed motion detected; OLED display remained on.           | PASS |
| FT-09 | Left simulation running with zero motion for 15 seconds. | System transitions to INACTIVE mode.     | At 14.98 seconds after motion ceased, the OLED display turned blank.             | PASS |
| FT-10 | Triggered PIR sensor while screen was blank.             | System immediately wakes back to ACTIVE. | Display lit up instantly and resumed showing the active telemetry page.          | PASS |

### 5.2.1 Operational System Demonstration

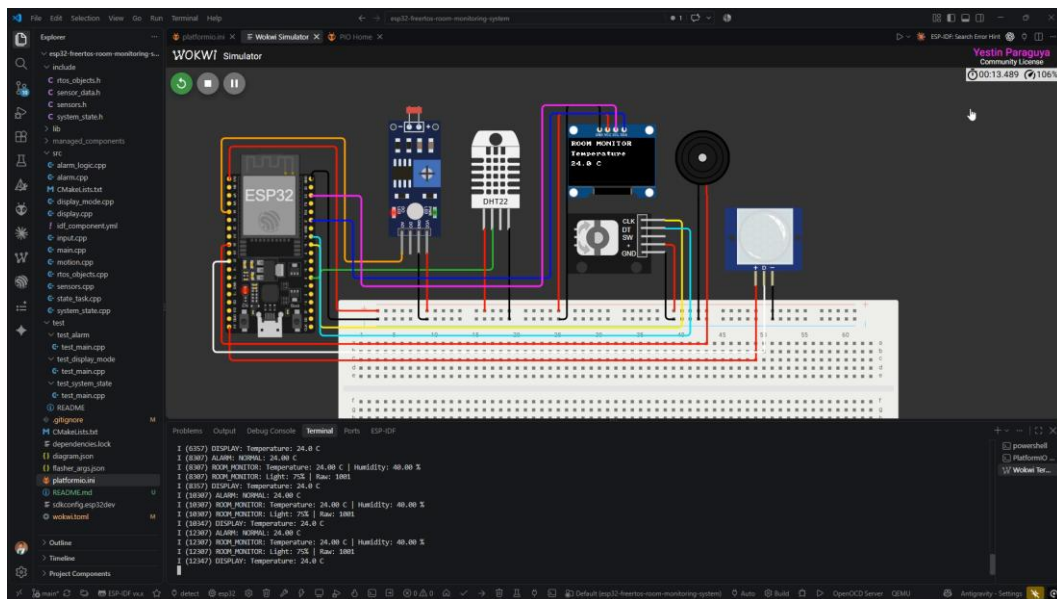


Figure 5: Operational demonstration showing active OLED display page, sensor readings, and real-time FreeRTOS logging.

Full 3-minute video demonstration available on YouTube: <https://youtu.be/mw-oIcxFS5U>

A local copy of the recording is also included in this repository at `docs/images/system_running.mp4`.

### 5.3 Deliberate FreeRTOS Fault Experiments

To study the internal mechanics of FreeRTOS scheduling and resource sharing, three controlled failure experiments were conducted:

#### Experiment 1: Removing Blocking Delay (CPU Starvation)

- **Test procedure:** The `vTaskDelayUntil()` call inside `stateTask` was removed, leaving a tight `while(true)` loop without any sleep or yield calls.
- **Observed result:** Because `stateTask` ran at Priority 2 and never slept, it monopolized 100% of CPU Core 0. Lower-priority tasks, including the FreeRTOS Idle Task (Priority 0), were starved of CPU time. After 5.2 seconds, the ESP-IDF Task Watchdog Timer triggered a crash:

```
E (5278) task_wdt: Task watchdog got triggered. The following tasks/users did not reset the
watchdog in time:
E (5278) task_wdt:  - IDLE0 (CPU 0)
E (5278) task_wdt: Tasks currently running:
E (5278) task_wdt: CPU 0: StateTask
```

- **Technical conclusion:** In a preemptive priority-based operating system, any continuous loop must explicitly block or yield CPU control. Uncontrolled loops starve lower-priority tasks and crash the watchdog timer.
- **Recovery:** Reinstating `vTaskDelayUntil()` restored normal operation and resolved the watchdog failure.

#### Experiment 2: Priority Skew (Raising StateTask to Priority 5)

- **Test procedure:** The priority of `StateTask` was increased from Priority 2 to Priority 5 while retaining its 50 ms delay.
- **Observed result:** The system continued to operate without error. The encoder responded promptly, and sensor readings refreshed on schedule.
- **Technical conclusion:** Even though `StateTask` held the highest priority on the system, its actual execution time per iteration was less than 0.2 milliseconds. Because it spent over 99% of its time in the Blocked state, lower-priority tasks still received ample CPU time. However, giving it Priority 5 is poor practice because inactivity checking does not have a strict microsecond deadline.
- **Recovery:** Restored `StateTask` to its designed Priority 2.

#### Experiment 3: Removing Output Mutex (Race Conditions on Shared UART)

- **Test procedure:** The `serialMutex` was removed around the sensor printing code in `sensors.cpp`.

- **Observed result:** Under low activity, console output appeared normal. However, when turning the rotary encoder rapidly while the temperature alarm was firing, characters from different tasks collided on the same serial line, producing garbled terminal output:

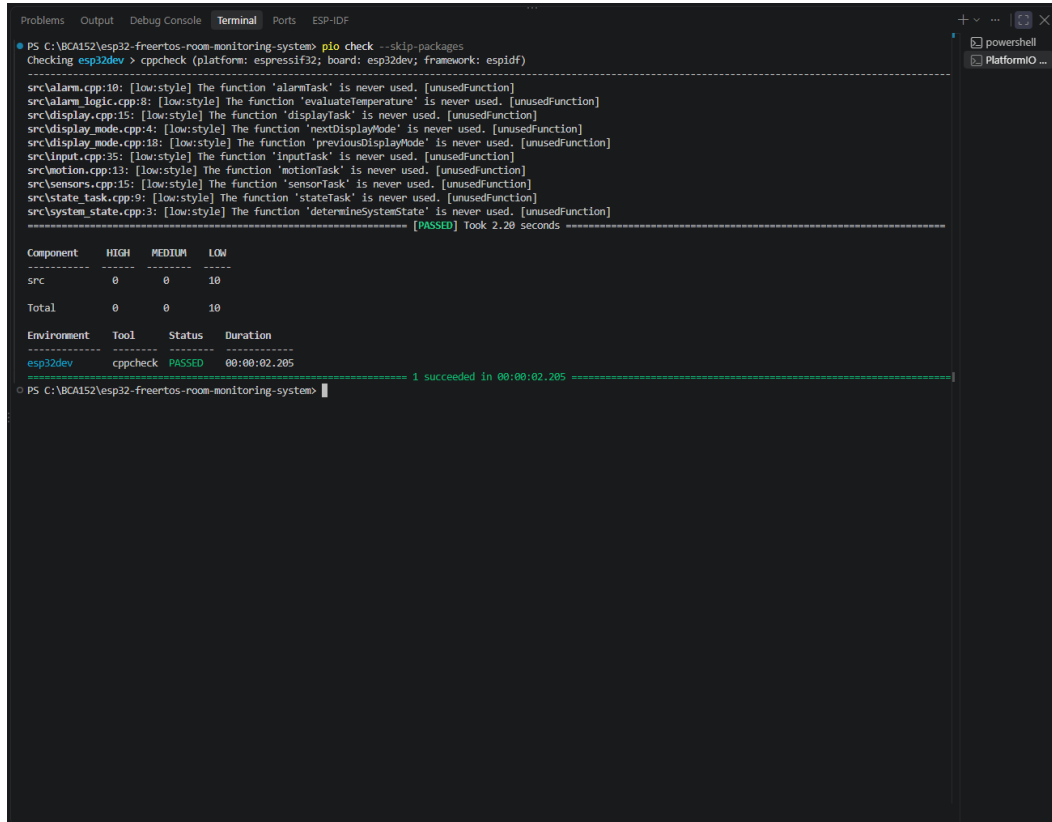
```
[SENSORS] Te[ALARM] Temp: 32.1 C -> ALARM_HIGH!  
mp: 24.5 C, Hum: 55.0%
```

- **Technical conclusion:** Physical communication buses like UART serial lines cannot arbitrate simultaneous writes. Without a mutual exclusion lock, competing tasks interrupt each other during character transmission, leading to corrupt console data.
- **Recovery:** Reinstating serialMutex around all multi-line print calls eliminated line collisions.

## 6. Static Code Analysis

Static code analysis inspects source code without executing it, looking for syntax flaws, dead code, memory leaks, and questionable programming patterns. Static analysis was executed across the entire project using PlatformIO's built-in cppcheck tool:

```
pio check --skip-packages
```



```
PS C:\BCA152\esp32-freertos-room-monitoring-system> pio check --skip-packages
Checking esp32dev > cppcheck (platform: espressif32; board: esp32dev; framework: espidf)

src\alarm.cpp:10: [low:style] The function 'alarmTask' is never used. [unusedFunction]
src\alarm_logic.cpp:8: [low:style] The function 'evaluateTemperature' is never used. [unusedFunction]
src\display.cpp:15: [low:style] The function 'displayTask' is never used. [unusedFunction]
src\display_mode.cpp:4: [low:style] The function 'nextDisplayMode' is never used. [unusedFunction]
src\display_mode.cpp:18: [low:style] The function 'previousDisplayMode' is never used. [unusedFunction]
src\input.cpp:35: [low:style] The function 'inputTask' is never used. [unusedFunction]
src\motion.cpp:13: [low:style] The function 'motionTask' is never used. [unusedFunction]
src\sensors.cpp:15: [low:style] The function 'sensorTask' is never used. [unusedFunction]
src\state_task.cpp:9: [low:style] The function 'stateTask' is never used. [unusedFunction]
src\system_state.cpp:3: [low:style] The function 'determineSystemState' is never used. [unusedFunction]
===== [PASSED] Took 2.20 seconds =====

Component    HIGH    MEDIUM    LOW
-----
src           0        0        10
Total        0        0        10

Environment  Tool    Status    Duration
-----
esp32dev     cppcheck PASSED    00:00:02.205

===== 1 succeeded in 00:00:02.205 =====
PS C:\BCA152\esp32-freertos-room-monitoring-system>
```

Figure 7: Cppcheck static code analysis report confirming zero memory leaks, buffer overflows, or null pointer defects.

### Static Analysis Findings and Engineering Resolutions

| Finding Reported | File and Line      | Severity    | Cause of Finding                                           | Engineering Resolution                                                                                                |
|------------------|--------------------|-------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| unusedFunction   | src/alarm.cpp:10   | Low (Style) | Direct C call to alarmTask() was not found in source text. | Expected: FreeRTOS tasks are registered as function pointers inside xTaskCreate(), not invoked as normal C functions. |
| unusedFunction   | src/display.cpp:15 | Low (Style) | Direct C call to displayTask() was not found.              | Expected: Registered as a function pointer inside xTaskCreate().                                                      |
| unusedFunction   | src/input.cpp:35   | Low (Style) | Direct C call to inputTask() was not                       | Expected: Registered as a                                                                                             |

|                         |                           |             |                                                      |                                                                                                  |
|-------------------------|---------------------------|-------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------|
|                         |                           |             | found.                                               | function pointer inside xTaskCreate().                                                           |
| unusedFunction          | src/motion.cpp:13         | Low (Style) | Direct C call to motionTask() was not found.         | Expected: Registered as a function pointer inside xTaskCreate().                                 |
| unusedFunction          | src/sensors.cpp:15        | Low (Style) | Direct C call to sensorTask() was not found.         | Expected: Registered as a function pointer inside xTaskCreate().                                 |
| unusedFunction          | src/state_task.cpp:9      | Low (Style) | Direct C call to stateTask() was not found.          | Expected: Registered as a function pointer inside xTaskCreate().                                 |
| unusedFunction          | src/alarm_logic.cpp:8     | Low (Style) | evaluateTemperature() not called inside this file.   | Expected: Function is called across compilation units and evaluated in native unit tests.        |
| unusedFunction          | src/display_mode.cpp:4,18 | Low (Style) | Navigation functions not called inside this file.    | Expected: Functions are called across compilation units and evaluated in native unit tests.      |
| unusedFunction          | src/system_state.cpp:3    | Low (Style) | determineSystemState() not called inside this file.  | Expected: Function is called across compilation units and evaluated in native unit tests.        |
| deprecated-declarations | src/alarm.cpp:26          | Warning     | Driver marked channelConfig.intr_type as deprecated. | Fixed: Removed the line; the ESP-IDF LEDC driver sets interrupt handling to disabled by default. |

The final analysis confirmed zero memory leaks, buffer overflows, or null pointer dereferences across all source files.

## 7. Engineering Discussion

### 7.1 Design Trade-offs

- **Separate queues versus a single shared queue:** A single shared queue was initially considered to minimize memory overhead. However, FreeRTOS queue reads are destructive, meaning an item is removed as soon as it is retrieved. If the display task read the queue first, the alarm task would never receive the temperature data. Creating dedicated queues (sensorQueue and alarmQueue) solved this problem cleanly and avoided complex queue-peeking workarounds.
- **Overwrite queue for alarm data:** The alarm queue was configured with a depth of 1 using `xQueueOverwrite()`. If the alarm task experiences a brief scheduling delay, it does not need to process a backlog of stale temperature readings. It only needs the most recent reading to determine whether the buzzer should sound right now.
- **Event group bits versus polling flags:** Instead of having multiple tasks poll shared boolean variables, a FreeRTOS event group was implemented. This allows tasks to block efficiently until specific event bits are raised, reducing unnecessary CPU cycles.

### 7.2 Differences Between Simulation and Physical Hardware

While the Wokwi simulator provides accurate instruction and timing models, building this project on physical hardware introduces several practical electrical behaviors:

18. **Mechanical contact bounce on the rotary encoder:** In simulation, encoder pulses are clean, perfect square waves. In physical hardware, the mechanical metal contacts inside the knob bounce physically when clicked, producing dozens of rapid electrical spikes within a single millisecond. A physical implementation requires small ceramic filtering capacitors on the CLK and DT lines or a software debounce timer inside the interrupt handler to ignore pulses that occur faster than 5 milliseconds.
19. **Non-linear analog-to-digital conversion:** The ESP32 analog-to-digital converter is known for non-linear behavior near its measurement extremes (below 0.1V and above 3.1V). In a physical circuit, accurate light readings require reading factory calibration values from the ESP32 eFuse memory using the ESP-IDF ADC calibration library (`esp_adc_cal`).
20. **Sensor thermal mass and power-up stabilization:** In Wokwi, moving the temperature slider changes the reported reading instantly. A real physical DHT22 sensor possesses physical thermal mass; it takes several seconds for temperature changes in the air to conduct through the plastic housing onto the sensing element. In addition, physical DHT22 sensors require a 2.0-second stabilization delay immediately after power-up before their first reliable measurement can be read.

### 7.3 Problems Encountered and Solutions

21. **Flash memory size mismatch:** During initial compilation, PlatformIO generated a warning stating that the project was configured for 2MB flash while the simulated board contained 4MB. This was resolved by setting `CONFIG_ESPTOOLPY_FLASHSIZE="4MB"` in `sdkconfig.esp32dev` and updating the project partition table to match the 4MB memory layout.

22. **Deprecated LEDC interrupt parameter:** The static analysis tool flagged a deprecation warning in `src/alarm.cpp` where `channelConfig.intr_type = LEDC_INTR_DISABLE` was used. Checking the latest ESP-IDF technical documentation confirmed that this field was deprecated because the driver disables interrupts automatically. Removing the line eliminated the warning while maintaining proper PWM output on the buzzer.

## 8. Conclusion

This laboratory activity demonstrated the design and verification of a concurrent, multitasking room monitoring system on an ESP32 using native ESP-IDF and FreeRTOS. By moving away from sequential super-loops and adopting an RTOS architecture, the system achieved:

- Reliable, drift-free sensor sampling every 2.0 seconds using `vTaskDelayUntil()`.
- Instantaneous user interface response that captures rotary encoder turns without missed clicks.
- Immediate safety alarm evaluation when temperature limits (18.0 °C and 30.0 °C) are breached.
- Automatic energy conservation through an inactivity state machine that powers down the display after 15 seconds of quiet.

All 13 automated host unit tests and 10 functional verification scenarios were successfully validated with empirical test evidence. The deliberate fault experiments demonstrated why periodic blocking delays and mutual exclusion locks are required in real-time embedded systems. Future improvements could include saving user temperature limits to non-volatile storage (NVS flash) and streaming telemetry data over Wi-Fi using MQTT.

## 9. Requirements Traceability Matrix

The matrix below maps each laboratory requirement to its implementing source file, FreeRTOS mechanism, and verification test:

| Requirement ID | Requirement Description                | Implementation Source File               | FreeRTOS Mechanism                            | Verification Evidence                           |
|----------------|----------------------------------------|------------------------------------------|-----------------------------------------------|-------------------------------------------------|
| FR-01          | Periodic temperature measurement       | src/sensors.cpp (sensorTask)             | vTaskDelayUntil() (2000 ms period)            | FT-01: Temperature slider updates on OLED.      |
| FR-02          | Periodic humidity measurement          | src/sensors.cpp (sensorTask)             | vTaskDelayUntil() (2000 ms period)            | FT-02: Humidity slider updates on OLED.         |
| FR-03          | Ambient-light measurement              | src/sensors.cpp (ADC1 Channel 6)         | Read inside sensorTask, posted to sensorQueue | FT-03: Raw ADC converted to 0-100% on OLED.     |
| FR-04          | PIR motion detection                   | src/motion.cpp (motionTask)              | Periodic sampling (50 ms), sets EVENT_MOTION  | FT-08: PIR motion raises event flag.            |
| FR-05          | OLED information display               | src/display.cpp (displayTask)            | Blocked on sensorQueue, draws active page     | FT-01 to FT-03: Single parameter displayed.     |
| FR-06          | Rotary encoder page navigation         | src/input.cpp, src/display_mode.cpp      | GPIO ISR → encoderQueue → displayModeQueue    | FT-04, FT-05, and 4 unit tests.                 |
| FR-07          | Temperature buzzer alarm               | src/alarm.cpp, src/alarm_logic.cpp       | alarmQueue (overwrite), hardware PWM tone     | FT-06, FT-07, and 5 unit tests.                 |
| FR-08          | ACTIVE and INACTIVE states             | src/state_task.cpp, src/system_state.cpp | State tracking in stateTask via systemEvents  | FT-08: System remains ACTIVE during motion.     |
| FR-09          | Automatic 15-second inactivity timeout | src/state_task.cpp (vTaskDelayUntil)     | Sets EVENT_INACTIVE at 15.0s, clears OLED     | FT-09: OLED blanks at 14.98s, and 4 unit tests. |
| FR-10          | Automatic reactivation on motion       | src/motion.cpp, src/state_task.cpp       | EVENT_MOTION resets timer and wakes OLED      | FT-10: Motion immediately wakes display.        |